Advanced Shell Features

1.1 Introducción

Aunque la mayoría de las distribuciones de Linux utilizan una interfaz gráfica de usuario (GUI), es más probable que los administradores trabajen en la interfaz de línea de comandos (CLI) para realizar tareas de rutina. La línea de comando normalmente tiene características mucho más poderosas, que no están disponibles en una GUI, como scripts (es decir, scripts de Shell) que los administradores pueden usar para realizar tareas remotas y avanzadas como configurar servicios del sistema.

En general, los administradores trabajan con muchos servidores simultáneamente y la interfaz GUI no se presta para este tipo de trabajo. Las GUI también utilizan valiosos recursos del sistema (como RAM y CPU), lo que ralentiza el rendimiento de otras funciones del servidor (como FTP y HTTP).

Como resultado, sentirse cómodo con el entorno de la línea de comandos es crucial para cualquiera que trabaje con Linux de forma regular. El enfoque principal de este módulo es aumentar su nivel de comodidad y productividad mientras trabaja con un entorno de carcasa.

Este capítulo tratará las características del shell y la programación del shell. Comprender cómo personalizar un entorno de shell es una habilidad esencial tanto para los usuarios finales como para los administradores. Los usuarios finales pueden querer modificar su entorno de shell para satisfacer sus necesidades específicas. Por ejemplo, un desarrollador de software puede querer un entorno diferente al de alguien que crea y mantiene sitios web.

Los administradores pueden hacer que el entorno de shell sea más fácil de usar para conjuntos completos de usuarios. Saber cómo personalizar un entorno de shell y aplicar estas personalizaciones a varias cuentas de usuario proporcionará un entorno de "mejores prácticas" más fácil de usar. Esto debería aumentar la productividad y reducir los problemas de soporte.

En este capítulo, aprenderá una variedad de funciones de personalización de shell, incluidas variables, alias y archivos de personalización.

1.2 Nota de certificación LPIC-1

Las personas que hayan utilizado el curso anterior para la preparación del examen LPIC-1 101 notarán una gran cantidad de redundancia en este capítulo. Muchos de los temas que se tratarán en este capítulo también se trataron en el curso anterior porque estos temas se pueden probar en ambos exámenes LPIC-1. Estas redundancias se destacan en **negrita** en la siguiente tabla :

Exam:	**LPIC-1 101**	**LPIC-1 102**
Topic:	• 103.1 Work on the command line	• 105.1 Customize and use the shell environment
Key Knowledge Areas:	• Use single shell commands and one-line command sequences to perform basic tasks on the command line. • **Use and modify the shell environment, including defining, referencing, and exporting environment variables.** • Use and edit command history. • **Invoke commands inside and outside the defined path.**	• **Set environment variables (e.g., PATH) at login or when spawning a new shell.** • Write BASH functions for frequently used sequences of commands. • Maintain skeleton directories for new user accounts. • **Set command search path with the proper directory.**
Terms and Utilities:	• .bash_history • bash • echo • **env** • exec • **export** • pwd • **set** • **unset** • man • uname • history • Quoting	• **.** • **./etc/profile** • **env** • **export** • **set** • unset • **~/.bash_profile** • **~/.bash_login** • **~/.profile** • **~/.bashrc** • **~/.bash_logout** • function • alias

While some of the topics covered in this chapter will review information covered in the previous course, there will also be new material introduced in this chapter.

1.3 Variables de Shell

Una variable es un nombre o identificador al que se le puede asignar un valor. El shell (y otros comandos) lee los valores de estas variables, lo que puede resultar en un comportamiento alterado dependiendo del contenido (valor) de la variable.

Las variables suelen tener un valor único, como `0` o `bob`. Algunos pueden contener varios valores separados por espacios como `Joe Brown` o por otros caracteres, como dos puntos: `/usr/bin:/usr/sbin:/bin:/usr/bin:/home/joe/bin`.

Usted asigna un valor a una variable escribiendo el nombre de la variable, seguido inmediatamente por un signo igual =y luego el valor. Por ejemplo: `name="Bob Smith"`.

Los nombres de las variables deben comenzar con una letra (carácter alfabético) o subrayado y contener solo letras, números y el carácter de subrayado. Es importante recordar que los nombres de las variables distinguen entre mayúsculas y minúsculas; `a` y `A` son variables diferentes. Al igual que con los argumentos de los comandos, se deben usar comillas simples o dobles cuando se incluyen caracteres especiales en el valor asignado a la variable para evitar la expansión del shell.

Valid Variable Assignments	Invalid Variable Assignments
`a=1`	`1=a`
`_1=a`	`a-1=3`
`LONG_VARIABLE='OK'`	`LONG-VARIABLE='WRONG'`
`Name='Jose Romero'`	`'user name'=anything`
`sshdirectory='/etc/ssh'`	`"sshdirectory"=etc/ssh`

El shell Bash y muchos comandos hacen un uso extensivo de variables. Uno de los principales usos de las variables es proporcionar un medio para configurar varias preferencias para cada usuario. El shell y los comandos de Bash pueden comportarse de diferentes maneras, según el valor de las variables. Para el shell, las variables pueden afectar lo que muestra el indicador, los directorios en los que el shell buscará comandos para ejecutar y mucho más.

1.4 Variables locales y ambientales

Una variable *local* solo está disponible para el shell en el que se creó. Una variable de *entorno* está disponible para el shell en el que se creó y se pasa a todos los demás comandos / programas iniciados por el shell.

Por convención, los caracteres en minúsculas se utilizan para crear nombres de variables locales y los caracteres en mayúsculas se utilizan al nombrar una variable de entorno. Por ejemplo, se puede llamar a una variable local `test`, mientras que se puede llamar a una variable de entorno `TEST`. Si bien esta es una convención que sigue la mayoría de la gente, no es una regla.

Hay varias formas de mostrar los valores de las variables. El `set` comando por sí solo mostrará todas las variables (locales y de entorno):

```
sysadmin@localhost:~$ set | head
BASH=/bin/bash
BASHOPTS=checkwinsize:cmdhist:complete_fullquote:expand_aliases:extglo
b:extquote
:force_fignore:histappend:interactive_comments:login_shell:progcomp:pr
omptvars:s
ourcepath
BASH_ALIASES=()
BASH_ARGC=()
BASH_ARGV=()
BASH_CMDS=()
BASH_COMPLETION_VERSINFO=([0]="2" [1]="8")
BASH_LINENO=()
BASH_SOURCE=()
BASH_VERSINFO=([0]="4" [1]="4" [2]="19" [3]="1" [4]="release" [5]="x86
_64-pc-lin
ux-gnu")
```

Para mostrar sólo las variables de entorno, hay varios comandos que proporcionan casi el mismo resultado: `env`, `declare -x`, `typeset -x` or `export -p`:

```
sysadmin@localhost:~$ env | tail
PWD=/home/sysadmin
HOME=/home/sysadmin
MAIL=/var/mail/sysadmin
SHELL=/bin/bash
TERM=xterm
```

```
SHLVL=1
LOGNAME=sysadmin
PATH=/home/sysadmin/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/
bin:/sbin:
/bin:/usr/games:/usr/local/games
LESSOPEN=| /usr/bin/lesspipe %s
_=/usr/bin/env
sysadmin@localhost:~$ declare -x | tail -5
declare -x PWD="/home/sysadmin"
declare -x SHELL="/bin/bash"
declare -x SHLVL="1"
declare -x TERM="xterm"
declare -x USER="sysadmin"
sysadmin@localhost:~$ typeset -x | tail -5
declare -x PWD="/home/sysadmin"
declare -x SHELL="/bin/bash"
declare -x SHLVL="1"
declare -x TERM="xterm"
declare -x USER="sysadmin"
sysadmin@localhost:~$ export -p | tail -5
declare -x PWD="/home/sysadmin"
declare -x SHELL="/bin/bash"
declare -x SHLVL="1"
declare -x TERM="xterm"
declare -x USER="sysadmin"
```

Para mostrar el valor de una variable específica, use el comando `echo` con el nombre de la variable prefijado por el signo `$` de dólar . Por ejemplo, para mostrar el valor de la variable `PATH`, ejecutaría `echo $PATH`.

```
sysadmin@localhost:~$ echo $PATH
/home/sysadmin/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/
sbin:/bin:
/usr/games:/usr/local/games
```

Considera esto

Las variables también se pueden incluir entre `{}` llaves para delimitarlas del texto circundante. Si bien el comando `echo ${PATH}` produciría el mismo resultado que el comando `echo $PATH`, las llaves distinguen la variable visualmente, lo que hace que sea más fácil de ver en los scripts en algunos contextos.

1.5 Creación de variables de entorno

De forma predeterminada, cuando se asigna una variable en el shell Bash, inicialmente se establece como una variable local. Hay algunas formas en que una variable local también puede convertirse en una variable de entorno. Primero, una variable local existente se puede *exportar* con el comando `export`:

```
sysadmin@localhost:~$ ENVIRONMENT_A=1
sysadmin@localhost:~$ export ENVIRONMENT_A
```

En segundo lugar, se puede *exportar* una nueva variable y asignarle un valor con un solo comando:

```
sysadmin@localhost:~$ export ENVIRONMENT_B=2
```

En tercer lugar, se puede utilizar el comando `declare` o `typeset` para *declarar* una variable como variable de entorno. Estos comandos son sinónimos y funcionan de la misma manera:

```
sysadmin@localhost:~$ declare -x ENVIRONMENT_C=3
sysadmin@localhost:~$ typeset -x ENVIRONMENT_D=3
```

Si todos los comandos anteriores se ejecutan, habrá cuatro nuevas variables de entorno:

```
sysadmin@localhost:~$ env | grep ENVIRONMENT
ENVIRONMENT_B=2
ENVIRONMENT_C=3
ENVIRONMENT_A=1
ENVIRONMENT_D=3
```

Con la siguiente sintaxis, el comando `env` también se puede usar para establecer temporalmente una variable:

```
env VARIABLE_NAME=TEMP_VALUE command
```

Los servidores a menudo se configuran en la hora universal coordinada (UTC), lo que es bueno para mantener una hora constante en los servidores de todo el

planeta, pero puede ser frustrante para el uso práctico de simplemente decir la hora:

```
sysadmin@localhost:~$ date
Fri Sep 18 21:27:25 UTC 2020
```

Para establecer temporalmente la variable de zona horaria `TZ`a la hora estándar del este (EST), use el `env`comando:

```
sysadmin@localhost:~$ env TZ=EST date
Fri Sep 18 16:27:54 EST 2020
```

La variable `TZ` se establece solo en el entorno del shell actual y solo durante la duración del comando `date`. El resto del sistema no se verá afectado por esta variable. De hecho, ejecutar el comando `date` nuevamente verificará que la variable `TZ` haya vuelto a UTC.

```
sysadmin@localhost:~$ date
Fri Sep 18 21:29:43 UTC 2020
```

1.6 Unsetting Variables

Si la opción de shell `nounset` está habilitada con el comando `set -o nounset` , entonces la referencia a una variable no configurada resultará en un `unbound variable`error. Cuando el comando se usa dentro de una secuencia de comandos, hacer referencia a una variable no configurada hará que la secuencia de comandos se cierre.

Esta opción puede *desactivarse* , usando el comando `set +o nounset`, donde cualquier referencia a una variable no configurada devolverá un valor nulo.

Si crea una variable y luego ya no desea que esa variable esté definida, use el comando `unset` para eliminarla:

```
sysadmin@localhost:~$ variable_1="This is a variable."
sysadmin@localhost:~$ echo $variable_1
This is a variable.
sysadmin@localhost:~$ unset variable_1
sysadmin@localhost:~$ set -o nounset
sysadmin@localhost:~$ echo $variable_1
-bash: variable_1: unbound variable
sysadmin@localhost:~$ set +o nounset
sysadmin@localhost:~$ echo $variable_1
```

```
sysadmin@localhost:~$
```

1.7 Comprensión de la variable PATH

La variable PATH es una de las variables de entorno más críticas para el shell, por lo que es importante comprender el efecto que tiene sobre cómo se ejecutarán los comandos.

El siguiente gráfico muestra una variable PATH típica , con nombres de directorio separados entre sí por dos puntos :

```
sysadmin@localhost:~$ echo $PATH
/home/sysadmin/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin
:/usr/bin:/sbin:/bin:
/usr/games:/usr/local/games
```

La variable PATH contiene una lista de directorios que se utilizan para buscar comandos ingresados por el usuario. Cuando el usuario escribe un comando y luego presiona la tecla **Enter** , PATH se busca en los directorios un archivo ejecutable que coincida con el nombre del comando. El procesamiento funciona a través de la lista de directorios de izquierda a derecha; el primer archivo ejecutable que coincide con lo que se escribe es el comando que el shell intentará ejecutar.

Considera esto

Antes de buscar la variable PATH para el comando, el shell primero determinará si el comando es un alias o una función, lo que puede resultar en que la variable PATH no se utilice cuando se ejecute ese comando específico.

Además, si el comando está integrado en el shell, la variable PATH no se utilizará.

Por último, si el usuario usa una ruta absoluta como /bin/ls o una ruta relativa como ./ls, entonces la variable PATH no se utiliza.

La siguiente tabla describe el propósito de algunos de los directorios que se muestran en la `PATH`variable anterior:

Directory	Contents
`/home/sysadmin/bin`	A directory for the current user `sysadmin` to place programs. Typically used by users who create their own scripts.
`/usr/local/sbin`	Normally empty, but may have administrative commands that have been compiled from local sources.
`/usr/local/bin`	Normally empty, but may have commands that have been compiled from local sources.
`/usr/sbin`	Contains the majority of the administrative command files.
`/usr/bin`	Contains the majority of the commands that are available for regular users to execute.
`/sbin`	Contains the essential administrative commands.
`/bin`	Contains the most fundamental commands that are essential for the operating system to function.

Para ejecutar comandos que no están contenidos en los directorios que se enumeran en la variable `PATH`, existen varias opciones:

- El comando se puede ejecutar escribiendo la ruta absoluta al comando.
- El comando se puede ejecutar con una ruta relativa al comando.
- La variabel `PATH`e se puede configurar para incluir el directorio donde se encuentra el comando.
- El comando se puede reubicar o copiar en un directorio que se enumera en la variable `PATH`.

Camino absoluto

Para demostrar *rutas absolutas* frente a *rutas relativas* , `my.sh`se crea un script ejecutable llamado en el directorio de inicio:

```
sysadmin@localhost:~$ echo 'echo Hello World!' > my.sh
sysadmin@localhost:~$ chmod u+x my.sh
```

Nota

El comando `chmod` se utiliza para modificar los permisos de los usuarios mediante el *método simbólico* . En el comando anterior, `u` representa al *propietario* del *usuario* , el `+` signo más representa *agregar el permiso* y `x` representa el `execute` permiso; en otras palabras, agregue el permiso de ejecución al propietario del usuario en el archivo.

Una ruta absoluta especifica la ubicación de un archivo o directorio desde el directorio de nivel superior a través de todos los subdirectorios hasta el archivo o directorio. Las rutas absolutas siempre comienzan con el `/`carácter que representa el directorio raíz. Por ejemplo, `/usr/bin/ls` es una ruta absoluta. Significa el *archivo ls, que está en el directorio bin, que está en el directorio usr, que está en el directorio / (raíz)* . Un archivo se puede ejecutar usando una ruta absoluta como esta:

```
sysadmin@localhost:~$ /home/sysadmin/my.sh
Hello World!
```

Relative Path

Una ruta relativa especifica la ubicación de un archivo o directorio en relación con el directorio actual. Por ejemplo, en el directorio `/home/sysadmin`, una ruta relativa de en `test/newfile` realidad haría referencia al archivo `/home/sysadmin/test/newfile`. Los caminos relativos nunca comienzan con el `/`.

El uso de una ruta relativa para ejecutar un archivo en el directorio actual requiere el uso del `.`carácter de punto, que simboliza el directorio actual:

```
sysadmin@localhost:~$ ./my.sh
Hello World!
```

Modify Variable `PATH`

Cuando un usuario quiere que su directorio de inicio se agregue a la variable `PATH` para ejecutar scripts y programas sin usar `./` delante del nombre del archivo, puede verse tentado a modificar la variable de ruta de la siguiente manera:

```
sysadmin@localhost:~$ echo $PATH
/home/sysadmin/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/
sbin:/bin:
/usr/games:/usr/local/games
sysadmin@localhost:~$ pwd
/home/sysadmin
sysadmin@localhost:~$ PATH=/home/sysadmin
```

Desafortunadamente, todo lo que anteriormente estaba contenido en la variable `PATH` se perderá.

```
sysadmin@localhost:~$ echo $PATH
/home/sysadmin
```

Esto se debe a que `PATH=/home/sysadmin` renuncia al valor de la variable `PATH`. Más adelante en esta sección se incluye un método correcto.

Los programas listados fuera del directorio `/home/sysadmin` ahora solo serán accesibles usando su ruta completa. Por ejemplo, suponiendo que el directorio de inicio aún no se haya agregado a la variable `PATH`, el comando `uname -a` se comportaría como se esperaba:

```
sysadmin@localhost:~$ uname -a
Linux localhost 4.4.0-72-generic #93~14.04.1-Ubuntu SMP Fri Mar 31 15:
05:15 UTC
2017 x86_64 x86_64 x86_64 GNU/Linux
```

Después de asignar el directorio de inicio a la `PATH`variable, el comando `uname -a` necesitará su nombre de ruta completo para ejecutarse correctamente:

```
sysadmin@localhost:~$ PATH=/home/sysadmin
sysadmin@localhost:~$ uname -a
Command 'uname' is available in '/bin/uname'
The command could not be located because '/bin' is not included in the
PATH envi
ronment variable.
uname: command not found
sysadmin@localhost:~$ /bin/uname -a
Linux localhost 4.4.0-72-generic #93~14.04.1-Ubuntu SMP Fri Mar 31 15:
05:15 UTC
2017 x86_64 x86_64 x86_64 GNU/Linux
```

Es posible agregar a la variable `PATH` sin sobrescribir su contenido anterior. Importe el valor actual de la variable `$PATH` en la variable recién definida `PATH` utilizándolo en ambos lados de la declaración de asignación.

Nota

Si cambió la variable `PATH` como lo hicimos en el ejemplo anterior y desea restablecerla, simplemente use el comando `exit` para cerrar la sesión:

```
sysadmin@localhost:~$ exit
```

Una vez que haya iniciado sesión, la variable PATH se restablecerá a su valor original.

```
sysadmin@localhost:~$ PATH=$PATH
```

Termine con el valor de la ruta del directorio de inicio adicional.

```
sysadmin@localhost:~$ PATH=$PATH:/home/sysadmin
sysadmin@localhost:~$ echo $PATH
/home/sysadmin/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/
sbin:/bin:/usr/games:/usr/local/games:/home/sysadmin
```

Ahora, los scripts ubicados en el directorio `/home/sysadmin` se pueden ejecutar sin usar una ruta:

```
sysadmin@localhost:~$ my.sh
Hello World!
```

Nota

En general, es una mala idea modificar la variable `PATH`. Si cambiara, los administradores lo verían como una actividad sospechosa. Las fuerzas malintencionadas quieren obtener privilegios elevados y acceso a información confidencial que reside en servidores Linux. Una forma de hacerlo es escribir un script que comparta el nombre de un comando del sistema y luego cambiar la variabel `PATH` para incluir el directorio de inicio del administrador. Cuando el administrador escribe el comando, ¡en realidad ejecuta el script malicioso!

Relocate the Command

A menudo es más sencillo copiar un script en un directorio que ya está incluido en el `PATH`. Por ejemplo, el usuario podría colocar el script en el direcotorio `/home/sysadmin/bin`, ya que ese directorio ya está incluido en la variable `PATH`.

Recuerde que es posible que diferentes comandos (ubicados en diferentes directorios) tengan el mismo nombre; el shell siempre intentará ejecutar el primer comando coincidente que encuentre. Por esta razón, es mejor evitar el uso de nombres para scripts que ya están usando otros comandos.

Se buscaran `PATH` los directorios enumerados en el orden en que se ingresen, de izquierda a derecha. Por ejemplo, según el resultado a continuación, si un script llamado `userupdate` está presente tanto en `usr/local/bin` el directorio `usr/local/games` como en el directorio, la versión que se encontraría y se ejecutaría sería la del directorio `usr/local/bin`.

```
sysadmin@localhost:~$ echo $PATH
/home/sysadmin/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/
sbin:/bin:/usr/games:/usr/local/games
```

Considera esto

¿Cómo puede saber si un comando que está creando podría tener el mismo nombre que otro comando en el sistema? Dado que el conflicto podría ser con un comando normal, otro script de shell, un comando incorporado, un alias o una función, podría ser un poco desafiante. Afortunadamente, el comando `type` le ayudará a descubrir posibles conflictos.

Como se demuestra en el resultado a continuación, el comando `type` puede determinar si el comando especificado ya existe. El comando final ejecutado en el ejemplo siguiente, usando el comando `junk`, ilustra cómo se ve cuando no existe tal comando:

```
sysadmin@localhost:~$ type echo
echo is a shell builtin
sysadmin@localhost:~$ type ls
ls is aliased to `ls --color=auto'
sysadmin@localhost:~$ type cal
cal is /usr/bin/cal
sysadmin@localhost:~$ type junk
-bash: type: junk: not found
```

1.8 Additional Common Variables

Es importante estar familiarizado con las variables de shell más utilizadas. Algunas de estas variables afectarán programas específicos, mientras que otras mostrarán información o cambiarán el comportamiento del shell. A continuación se describen algunas de las variables más importantes.

Prompt Variables

Normalmente, la solicitud incluye el nombre de usuario, el nombre de host y el directorio actual:

```
sysadmin@localhost:~$
```

Este indicador se puede cambiar modificando la `PS1`variable (Prompt Shell 1). Antes de realizar cambios en el `PS1`, haga una copia de seguridad con una variable local para que pueda restaurarse a su estado original:

```
sysadmin@localhost:~$ echo $PS1
\[\e]0;\u@\h: \w\a\]${debian_chroot:+($debian_chroot)}\[\033[01;32m\]\
u@\h\[\033
[00m\]:\[\033[01;34m\]\w\[\033[00m\]\$
sysadmin@localhost:~$ ps1=$PS1
```

Una vez que `PS1` se modifique, los cambios en el mensaje entrarán en vigencia inmediatamente, como se ve en el siguiente ejemplo:

```
sysadmin@localhost:~$ ps1=$PS1
sysadmin@localhost:~$ PS1="Command> "
Command> echo 'Hello World!'
Hello World!
Command>
```

There are several useful special character sequences available to customize this prompt. These special characters start with a backslash character. For example, to display the current path in the prompt, use either the `\w` or `\W` character sequence:

```
sysadmin@localhost:~$ cd /usr/share/doc
sysadmin@localhost:/usr/share/doc$ PS1="\w"
/usr/share/doc PS1="\W"
doc cd ~
~ PS1=$ps1
sysadmin@localhost:~$
```

Hay varias secuencias de caracteres especiales útiles disponibles para personalizar este mensaje. Estos caracteres especiales comienzan con una barra invertida. Por ejemplo, para mostrar la ruta actual en el indicador, use la secuencia de caracteres `\w`o `\W`:

Tenga en cuenta que `\w` proporciona una ruta completa en la solicitud `(/usr/share/doc` en el ejemplo anterior) mientras que `\W` proporciona una ruta relativa en la solicitud. Para ver las posibles secuencias de caracteres del mensaje, consulte la página de manual `bash` y busque `PROMPTING`:

```
sysadmin@localhost:~$ man bash

Output Omitted...

OPTIONS

       All of the single-character shell options documented in the des
cription

       of the set builtin command can be used as options  when  the  s
hell  is

/PROMPTING_
```

Presione la combinación de teclas **Esc + n** un par de veces para ver el resultado exacto como se muestra:

```
PROMPTING

       When executing interactively, bash displays the primary prompt
PS1 when

       it is ready to read a command, and the secondary prompt  PS2  w
hen  it

       needs  more  input  to  complete  a  command.  Bash allows thes
e prompt
```

```
       strings to be customized by inserting  a  number  of  backslash
-escaped

       special characters that are decoded as follows:

              \a     an ASCII bell character (07)

              \d     the  date  in "Weekday Month Date" format (e.g.,
"Tue May

                     26")


              \D{format}

                     the format is passed to strftime(3)  and  the  re
sult  is

                     inserted  into the prompt string; an empty format
results

                     in a locale-specific time representation.  The br
aces are

                     required

              \e     an ASCII escape character (033)

              \h     the hostname up to the first `.'

              \H     the hostname

              \j     the number of jobs currently managed by the shell

              \l     the basename of the shell's terminal device name

              \n     newline

              \r     carriage return

              \s     the  name  of  the shell, the basename of $0 (the
portion

                     following the final slash)
```

Tenga en cuenta que los cambios realizados en el indicador solo afectan al shell actual. Cerrar el shell y abrir uno nuevo resultará en un retorno al indicador original.

Nota

Para hacer cambios permanentes a una variable, defínalo en un archivo de inicialización.

1.8.2 History Variables

Varias variables afectan el uso del comando `history` (y funciones asociadas). Recuerde que el comando `history` se puede utilizar para *volver a ejecutar* comandos ejecutados anteriormente. La variable `HISTSIZE` determinará cuántos comandos mantener en la memoria para cada shell Bash.

Cuando se cierra el programa de shell, toma los comandos almacenados en la memoria y los guarda en el *archivo histórico* , que es el `~/.bash_history` predeterminado. Si un usuario desea almacenar los comandos del historial en un archivo diferente, el usuario puede especificar una ruta absoluta como valor para la variable local `HISTFILE`l:

```
HISTFILE=/tmp/history
```

De forma predeterminada, se almacenarán quinientos comandos en el archivo de historial. La variable `HISTFILESIZE` determinará cuántos comandos escribir en este archivo.

Si los valores de `HISTSIZE` y `HISTFILESIZE` son diferentes, el menor de los dos números es el número de comandos que realmente se escribirán en el archivo.

Aunque normalmente no tiene ningún valor predeterminado, puede ser ventajoso establecer un valor para la variable `HISTCONTROL` en un archivo de inicialización como el archivo `~/.bash_profile`. La variable `HISTCONTROL` puede establecer en cualquiera de las siguientes características:

- `ignoredups`: Esto evitará comandos duplicados que se ejecuten consecutivamente.
- `ignorespace`: Esto no almacenará ningún comando que comience con un espacio. Esto proporciona al usuario una manera fácil de ejecutar un comando que no entrará en la lista del historial.
- `ignoreboth`: Esto no almacenará duplicados consecutivos ni ningún comando que comience con un espacio.
- `erasedups`: Esto no almacenará un comando que sea idéntico a otro comando en su historial (en realidad, la entrada anterior del comando se eliminará de la lista del historial).
- `ignorespace:erasedups`: Esto incluirá el beneficio de `erasedups` con la ventaja de `ignorespace`.

Otra variable que afectará lo que se almacena en el historial de comandos es la variable `HISTIGNORE`e. La mayoría de los usuarios no quieren la lista del historial abarrotada de comandos básicos como `ls`, `cd`, `exit`, y `history`. La variable `HISTIGNORE` se puede usar para decirle a Bash que no almacene ciertos comandos en la lista del historial.

Para que los comandos no se incluyan en la lista del historial, incluya un comando como el siguiente en el archivo `~/.bash_profile`:

```
HISTIGNORE='ls*:cd*:history*:exit'
```

Considera esto

Recuerde que el `*`carácter de asterisco se usa para indicar *cualquier otra cosa después del comando* ; por lo tanto, `ls*`coincidiría con cualquier comando `ls`, como `ls -l`o `ls /etc`.

1.9 Alias

Un alias es esencialmente un apodo para un comando o una serie de comandos. Algunos alias pueden crearse automáticamente como resultado de comandos ejecutados en shells de inicialización. Para ver los alias actuales del shell, use el comando `alias` sin argumentos:

```
sysadmin@localhost:~$ alias
alias alert='notify-send --urgency=low -i "$([ $? = 0 ] && echo termin
al || echo
 error)" "$(history|tail -n1|sed -e '\''s/^\s*[0-9]\+\s*//;s/[;&|]\s*a
lert$//'\'
')"'
alias egrep='egrep --color=auto'
alias fgrep='fgrep --color=auto'
alias grep='grep --color=auto'
alias l='ls -CF'
alias la='ls -A'
alias ll='ls -alF'
alias ls='ls --color=auto'
```

Cada línea describe un alias en el formato en el que `alias` se ejecuta el comando al crear nuevos alias. Por ejemplo, el alias que define el comando de *lista larga* `ll` es:

```
alias ll='ls -alF'
```

Curiosamente, incluso el comando `ls` es un alias para sí mismo con soporte de color:

```
alias ls='ls --color=auto'
```

Los propósitos principales de crear un alias incluyen:

- **Para incluir una opción de comando por defecto:**
    - Por ejemplo, si la `-i` opción siempre se usa al ejecutar el comando `rm`, se puede crear un alias para `rm` que siempre se ejecute como `rm -i`.

```
• sysadmin@localhost:~$ alias rm='rm -i'
```

Para crear una forma corta de un comando largo:

- Por ejemplo, si el comando `ls -ld .*` se usa con frecuencia, se puede crear un alias para esto, quizás llamado `hidden`, que mostrará solo los archivos ocultos en un directorio. Cuando `hidden` se ejecuta como un comando, se ejecutará `ls -ld .*`

```
• sysadmin@localhost:~$ alias hidden="ls -ld .*"
• sysadmin@localhost:~$ hidden
• drwxr-xr-x 1 sysadmin sysadmin 4096 Sep 20 22:49 .
• drwxr-xr-x 1 root     root     4096 May  2 16:18 ..
• -rw------- 1 sysadmin sysadmin  194 Sep 20 21:58 .bash_history
• -rw-r--r-- 1 sysadmin sysadmin  220 Apr  4  2018 .bash_logout
• -rw-r--r-- 1 sysadmin sysadmin 3768 Apr 24 16:24 .bashrc
• drwx------ 2 sysadmin sysadmin 4096 Sep 20 21:54 .cache
• -rw------- 1 sysadmin sysadmin   51 Sep 20 22:49 .lesshst
• -rw-r--r-- 1 sysadmin sysadmin  807 Apr  4  2018 .profile
• -rw-r--r-- 1 sysadmin sysadmin   74 Apr 24 16:24 .selected_edito
  r
• -rw-r--r-- 1 sysadmin sysadmin    0 Apr 24 16:24 .sudo_as_admin_
  successful
```

Para crear comandos equivalentes a DOS:

- Mucha gente necesita trabajar tanto en sistemas Microsoft Windows como Linux. Las diferencias sutiles, como ejecutar `cls` (DOS) en lugar de `clear`(Linux) para limpiar la pantalla del terminal, pueden crear dolores de cabeza. En Linux, cree un alias llamado `cls` que ejecutará el comando `clear`.

```
• sysadmin@localhost:~$ cls
• -bash: cls: command not found
• sysadmin@localhost:~$ alias cls=clear
• sysadmin@localhost:~$ cls
```

- Los alias que se crean en la línea de comandos son específicos del shell en el que se crean. Si se abre un nuevo shell, los alias que se crean en este shell no existirán en el nuevo shell.
- Para que los alias sean permanentes, el comando `alias` debe colocarse en un shell de inicialización. Esta técnica se discutirá más adelante en este capítulo.
- Para eliminar un alias del shell actual, use el comando `unalias`:

```
sysadmin@localhost:~$ alias | grep hidden
alias hidden='ls -ld .*'
sysadmin@localhost:~$ unalias hidden
sysadmin@localhost:~$ alias | grep hidden
sysadmin@localhost:~$ hidden
hidden: command not found
```

Nota

Si el valor de alias (como `ls -l .*`) tiene espacios, asegúrese de encerrarlo entre comillas.

En algunos casos, es posible que un administrador simplemente desee evitar un alias temporalmente. Por ejemplo, considere una situación en la que el comando `rm` tiene un alias con el comando `rm -i`:

```
sysadmin@localhost:~$ alias rm='rm -i'
sysadmin@localhost:~$ rm -r Documents
rm: descend into directory 'Documents'? y
rm: remove regular file 'Documents/animals.txt'? n
rm: remove regular file 'Documents/alpha-third.txt'? n
rm: remove regular file 'Documents/longfile.txt'? n
rm: remove regular file 'Documents/red.txt'? n
rm: remove regular file 'Documents/alpha.txt'? n
rm: remove regular file 'Documents/hidden.txt'? n
rm: remove regular file 'Documents/food.txt'? n
rm: remove regular file 'Documents/adjectives.txt'? n
rm: remove regular file 'Documents/spelling.txt'? n
rm: remove regular file 'Documents/os.csv'? n
rm: remove regular file 'Documents/letters.txt'? n
rm: remove regular file 'Documents/people.csv'? n
rm: remove regular file 'Documents/letters.txt'? n
rm: remove regular file 'Documents/people.csv'? n
rm: remove regular file 'Documents/alpha-first.txt'? n
rm: remove regular file 'Documents/newhome.txt'? n
rm: remove regular file 'Documents/numbers.txt'? n
rm: remove regular file 'Documents/alpha-second.txt'? n
rm: descend into directory 'Documents/School'? n
rm: remove regular file 'Documents/profile.txt'? n
rm: remove regular file 'Documents/linux.txt'? n
rm: descend into directory 'Documents/Work'? n
```

```
rm: remove write-protected regular file 'Documents/words'? n
rm: remove regular file 'Documents/hello.sh'? n
sysadmin@localhost:~$
```

La opción `-i` le dice al coamndo `rm` que solicite cada archivo y directorio antes de eliminarlo. Esto puede resultar muy engorroso en estructuras de directorios grandes. Para evitar este alias para una sola ejecución del comando, use el `\`carácter de barra invertida antes del comando:

```
sysadmin@localhost:~$ mkdir test
sysadmin@localhost:~$ ls
Desktop    Downloads  Pictures  Templates  my.sh
Documents  Music      Public    Videos     test
sysadmin@localhost:~$ \rm -r test
sysadmin@localhost:~$ ls
Desktop    Downloads  Pictures  Templates  my.sh
Documents  Music      Public    Videos
```

Los alias también se pueden evitar utilizando una ruta absoluta o relativa al comando.

1.10 Funciones

Las funciones son un poco más avanzadas que los alias y normalmente se utilizan en los scripts de shell de Bash. Como resultado, se presentarán en esta unidad y se explorarán con mayor detalle en el próximo capítulo sobre secuencias de comandos de shell.

Una función se parece mucho a un alias en el sentido de que proporciona un *apodo* para otra cosa que requiere ejecución. Sin embargo, las funciones suelen estar diseñadas para ejecutar varios comandos, no un solo comando como un alias (aunque es posible ejecutar varios comandos con alias).

Para aquellos familiarizados con Java, C ++ o varios otros lenguajes de programación, esto les resultará muy familiar. Para crear una función, use la siguiente sintaxis:

```
function_name ()
{
   commands_here
}
```

En el ejemplo anterior, `function_name` puede ser cualquier cosa que desee el administrador. También se pueden usar muchos comandos para reemplazar el `commands_here` marcador de posición. Tenga en cuenta el formato, en particular, la ubicación de los paréntesis `()` y las llaves `{}`, así como la convención de usar pestañas para hacer que la función sea más fácil de leer.

Considere el escenario en el que un administrador ejecuta constantemente los siguientes comandos:

```
cd /usr/share/doc
echo "Document directory usage report" > /tmp/report
date >> /tmp/report
pwd >> /tmp/report

du -sh . >> /tmp/report
cd ~
```

En lugar de escribir estos comandos repetidamente, el administrador puede crear una función que ejecutará cada comando, uno a la vez:

```
sysadmin@localhost:~$ report () {
cd /usr/share/doc
echo "Document directory usage report" > /tmp/report
date >> /tmp/report
pwd >> /tmp/report
du -sh . >> /tmp/report
cd ~
}
sysadmin@localhost:~$ report
sysadmin@localhost:~$ more /tmp/report
Document directory usage report
Fri Sep 20 23:01:03 UTC 2019
/usr/share/doc
15M     .
```

Los `>` caracteres que ve en la salida anterior se denominan *avisos secundarios* . El shell Bash sabe que para finalizar la función, `}`se necesita una llave de cierre , por lo que solicita más información cada vez que se presiona la tecla **Intro** , hasta que se encuentra una llave de cierre.

Una buena mejora que se puede hacer a esta función es hacerla más flexible y permitir que el usuario pase un argumento a la función; este argumento representaría el directorio en el que el usuario desea ejecutar el comando `du`. Cuando se pasan argumentos a una función, se almacenan en variables especiales: `$1`para el primer argumento, `$2`para el segundo argumento, etc.

Por ejemplo, utilice las flechas para modificar el primer comando de la función de informe para permitir que el usuario pase un argumento de en qué directorio realizar las operaciones:

```
sysadmin@localhost:~$ report () { cd $1; echo "Document directory usag
e report"
> /tmp/report; date >> /tmp/report; pwd >> /tmp/report; du -sh . >> /t
mp/report;cd ~; }
sysadmin@localhost:~$ report /usr/share
sysadmin@localhost:~$ cd /usr/share
sysadmin@localhost:/usr/share$ more /tmp/report
Document directory usage report
Fri Sep 20 23:01:03 UTC 2019
/usr/share/doc
15M     .
sysadmin@localhost:/usr/share$ cd
sysadmin@localhost:~$
```

Nota

Nota Al igual que los alias, las funciones solo existen en el shell en el que se crean. Para que sean *permanentes* , colóquelas en un archivo de inicialización.

1.11 Listas

En el contexto del shell Bash, una lista es una secuencia de comandos que están separados por uno de los siguientes operadores:

Operador	Sentido
`;`	Los comandos dentro de la lista se ejecutan secuencialmente, donde el shell ejecutará el primer comando y esperará a que termine antes de ejecutar el siguiente comando. El estado de salida de la lista se basa en el estado de salida del último comando que se ejecuta.
`&`	Cada comando dentro de la lista se ejecuta de forma asíncrona dentro de una subcapa o en segundo plano. El shell no espera a que terminen los comandos y devuelve un estado de salida de cero.
`&&`	Esta es una lista AND, por lo que si el comando del lado izquierdo se `&&`ejecuta correctamente, se ejecutará el comando del lado derecho `&&`. El estado de salida de la lista se basa en el estado de salida del último comando que se ejecuta.
`||`	Esta es una lista OR, por lo que si el comando del lado izquierdo de `||`NO se ejecuta correctamente, se ejecuta el comando del lado derecho de `||`. El estado de salida de la lista se basa en el estado de salida del último comando que se ejecuta.

Para asegurarse de que cada comando se ejecute en orden, use una lista secuencial simple con los comandos separados por el `;` carácter de punto y coma . Por ejemplo, para ejecutar el coamndo `date,` seguido del comando `who` y luego el comando `uptime`, ejecute algo como:

```
sysadmin@localhost:~$ date;who;uptime
Fri Sep 18 23:09:31 UTC 2020
sysadmin console      Sep 18 21:58
 23:09:31 up 9 days, 21:50,  1 user,  load average: 0.30, 0.48, 0.45
```

Si la lista de comandos que un administrador desea ejecutar tardará en completarse y el administrador desea recuperar el acceso al shell de inmediato, los comandos se pueden ejecutar de forma asincrónica en segundo plano como una lista como esta:

```
sysadmin@localhost:~$ updatedb& makewhatis&
[1] 3259
[2] 3260
```

El coamndo `updatedb` actualiza la base de datos utilizada por el comando `locate`, mientras que el comando `makewhatis` recopila información de las páginas de manual y la indexa en una base de datos. Tenga en cuenta que el comando `makewhatis` no existe en nuestra VM; por lo tanto, es posible que la salida del ejemplo anterior no coincida con la salida de nuestro entorno virtual.

Observe que el shell generará un número de trabajo entre corchetes, junto con un número de identificación de proceso (PID) para cada proceso en segundo plano. Estos números pueden ser utilizados por comandos como `kill` y otros para controlar estos procesos o trabajos.

La lista AND forma una cadena condicional de comandos separados por los `&&`caracteres dobles y comerciales . Solo cuando el comando en el lado izquierdo `&&`de la `&&`ejecución tenga éxito, el comando en el lado derecho de la ejecución. Si un usuario quisiera copiar un archivo después de crear un directorio, siempre que el directorio no existiera previamente, podría ejecutar la siguiente cadena de comandos:

```
sysadmin@localhost:~$ mkdir ./Backup && cp /etc/hosts ./Backup
sysadmin@localhost:~$ ls
Backup   Documents  Music     Public     Videos
Desktop  Downloads  Pictures  Templates  my.sh

sysadmin@localhost:~$ ls ./Backup/
hosts
```

Si el primer comando tiene éxito, se ejecutará el segundo comando. El resultado será una nueva carpeta llamada `Backup` con una copia del archivo `/etc/hosts`. Si el mismo comando se ejecutó en un directorio sin los permisos adecuados, la falla del comando `mkdir` haría que el comando `cp` no se ejecutara.

```
sysadmin@localhost:~$ cd /usr
sysadmin@localhost:/usr$ mkdir ./Backup && cp /etc/hosts ./Backup
mkdir: cannot create directory './Backup': Permission denied
```

Usando una lista OR, cuando se ejecuta un comando, otro comando se ejecutará solo si falla. En este ejemplo, el comando `test` se usa para ver si `$HOME/bin` existe un directorio . Si esta prueba falla, se crea el directorio:

```
sysadmin@localhost:~$ test -e $HOME/bin || mkdir $HOME/bin
sysadmin@localhost:~$ ls $HOME
Backup   Documents  Music     Public     Videos  my.sh
Desktop  Downloads  Pictures  Templates  bin
```

Aunque los ejemplos proporcionados de listas de comandos, hasta ahora, han sido todos donde todos los elementos de las listas estaban separados por el mismo operador, esto se ha hecho para mantener los ejemplos simples. Se pueden construir listas que combinen los distintos operadores de formas interesantes y poderosas, especialmente combinando los tipos de lista AND y OR. Por ejemplo, la siguiente lista Y / O híbrida podría usarse dentro de un script para asegurarse de que la variable nombrada `search` se establezca en el primer argumento pasado al script o, de lo contrario, se establezca en el directorio de trabajo actual:

```
test $# -eq 0 && search=`pwd` || search=$1
```

1.12 Archivos de inicialización

Cuando un usuario abre un nuevo shell, ya sea durante el inicio de sesión o cuando ejecuta una terminal que inicia un shell, el shell se personaliza mediante archivos llamados archivos de inicialización (o configuración). Estos archivos de inicialización establecen el valor de las variables, crean alias y funciones y ejecutan otros comandos que son útiles para iniciar el shell.

Hay dos tipos de archivos de inicialización: archivos de inicialización *global* que afectan a todos los usuarios del sistema y archivos de inicialización *locales* que son específicos de un usuario individual.

Los archivos de configuración global se encuentran en el directorio `/etc`. Los archivos de configuración local se almacenan en el directorio de inicio del usuario.

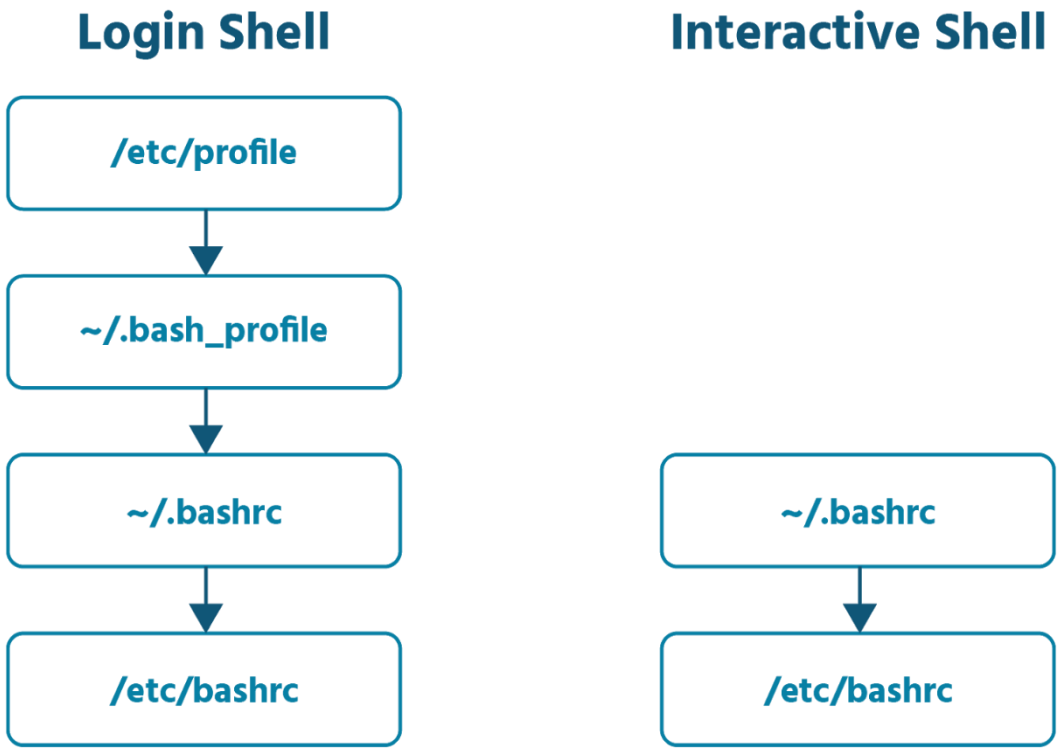
Archivos de inicialización BASH

Cada shell usa diferentes archivos de inicialización. Además, la mayoría de los shells ejecutan diferentes archivos de inicialización cuando el shell se inicia a través del proceso de inicio de sesión (llamado shell de inicio de sesión) frente a cuando un terminal inicia un shell (llamado shell sin inicio de sesión o shell interactivo).

El siguiente diagrama ilustra los diferentes archivos que se inician con un shell de inicio de sesión típico frente a un shell interactivo:

Nota

El `~`carácter de tilde representa el directorio de inicio del usuario y los nombres de archivo precedidos por un `.`carácter de punto indican archivos ocultos.



Cuando Bash se inicia como un shell interactivo, ejecuta el archivo `~/.bashrc`, que también puede ejecutar el archivo `/etc/bashrc`, si existe. Nuevamente, dado que el archivo `~/.bashrc` es propiedad del usuario que inicia sesión, el usuario puede evitar la ejecución del archivo `/etc/bashrc`.

Con tantos archivos de inicialización, una pregunta común en este momento es: "¿Qué archivo se supone que debo usar?" El siguiente cuadro ilustra el propósito de cada uno de estos archivos, proporcionando ejemplos de los comandos que puede colocar en cada archivo:

Fichero	Propósito
`/etc/profile`	Este archivo solo puede ser modificado por el administrador y lo ejecutarán todos los usuarios que inicien sesión. Los administradores utilizan este archivo para crear variables de entorno clave, mostrar mensajes a los usuarios cuando inician sesión y establecer valores clave del sistema.
`~/.bash_profile`	Cada usuario tiene su propio archivo `.bash_profile` en su directorio de inicio. El propósito de este archivo es el mismo que el del archivo `/etc/profile`, pero tener este archivo permite al usuario personalizar el shell a sus propios gustos. Este archivo se utiliza normalmente para crear variables de entorno personalizadas.
`~/.bashrc`	Cada usuario tiene su propio archivo `.bashrc` en su directorio de inicio. El propósito de este archivo es generar elementos que deben crearse para cada shell, como variables locales y alias.
`/etc/bashrc`	Este archivo puede afectar a todos los usuarios del sistema. Solo el administrador puede modificar este archivo. Al igual que el archivo `.bashrc`, el propósito de este archivo es generar elementos que deben crearse para cada shell, como variables locales y alias.

1.13 Modificación de archivos de inicialización

La forma en que opera el shell de un usuario se puede cambiar modificando los archivos de inicialización de ese usuario. La modificación de la configuración global requiere acceso administrativo al sistema, ya que los archivos del directorio `/etc` solo pueden ser modificados por un administrador. Un usuario solo puede modificar los archivos de inicialización en su directorio de inicio.

La mejor práctica es crear una copia de seguridad antes de modificar los archivos de configuración como seguro contra errores; una copia de seguridad siempre se puede restaurar si algo sale mal.

En algunas distribuciones de Linux, el valor predeterminado `~/.bash_profile` contiene las siguientes dos líneas que personalizan la variable `PATH` de entorno:

```
PATH=$PATH:$HOME/bin

export PATH
```

La primera línea establece la variable `PATH` en el valor existente de la variable `PATH` con la adición del subdirectorio bin del directorio de inicio del usuario. La variable `$HOME` se refiere al directorio de inicio del usuario. Por ejemplo, si el usuario que inicia sesión es *joe* , entonces `$HOME/bin` es `/home/joe/bin`.

La segunda línea convierte la varianle `PATH` local en una variable de entorno.

El archivo `~/.bashrc` predeterminado ejecuta /etc/bashrc usando una declaración como

```
. /etc/bashrc
```

El `.`carácter de punto se utiliza para *obtener* un archivo con el fin de ejecutarlo. El `.`carácter de punto también tiene un comando sinónimo, el coamndo `source`, pero el uso del `.`carácter de punto es más común que usar el comando `source`.

El aprovisionamiento puede ser una forma eficaz de probar los cambios realizados en los archivos de inicialización, lo que le permite corregir cualquier error sin tener que cerrar la sesión y volver a iniciarla. Si actualiza `~/.bash_profile` para modificar la variable `PATH`, puede verificar que sus cambios son correctos ejecutando lo siguiente:

```
. ~/.bash_profile

echo $PATH
```

1.14 Scripts de salida de BASH

Así como Bash ejecuta uno o más archivos al iniciarse, también puede ejecutar uno o más archivos al salir. Cuando Bash salga, ejecutará los
s archivos `~/.bash_logout`y `/etc/bash_logout`, si existen. Normalmente, estos archivos se utilizan para *limpiar* tácticas cuando el usuario sale del shell. Por ejemplo, el predeterminado `~/.bash_logout`ejecuta el comando `clear` para eliminar cualquier texto presente en la pantalla del terminal.

Considera esto

Cuando se crea un nuevo usuario, el administrador puede copiar archivos automáticamente desde `/etc/skel` el directorio de inicio del usuario. Estos archivos ayudan a configurar un entorno útil a través de archivos de inicio que se leen cuando el usuario inicia sesión en su cuenta.

Junto con los archivos que se proporcionan cuando el usuario inicia sesión en su cuenta, los archivos que controlan las tareas que suceden cuando el usuario cierra la sesión se proporcionan a las cuentas desde el directorio `/etc/skel` cuando se crea la cuenta. Por lo general, estas tareas limpian el entorno que utilizó la cuenta del usuario durante la sesión de inicio de sesión.

Cuando se crea un nuevo usuario, los archivos del directorio `/etc/skel` se copian automáticamente en el directorio de inicio del nuevo usuario. Como administrador, puede modificar los archivos en el directorio `/etc/skel` para proporcionar funciones personalizadas a los nuevos usuarios.

Lab 1: Advanced Shell Features

1.0 Introduction

In this lab, students will gain practice in customizing the shell and simple shell programming by performing the following tasks with CentOS:

1. View and modify local and environment variables to customize the shell.
2. Create, use, and avoid aliases for executing commands within the shell.
3. Define functions that execute multiple commands.
4. Use lists of commands to logically execute commands.
5. Customize the bash initialization files.
6. Create several simple bash scripts.

1.1 Step 1

For this lab you will be using the CentOS PC virtual machine.

Press the **Enter** key display the prompt in the virtual machine.

```
This lab has two user accounts (username :: password )


   root     :: vagrant
   vagrant :: vagrant


Press the [Enter] key to begin...



[sysadmin@localhost ~]$
```

Recall that shell variables contain useful information for the shell and other executable programs. Display all variables (both local and environment) by executing the `set` command:

```
set
```

```
[sysadmin@localhost ~]$ set
BASH=/bin/bash
BASHOPTS=checkwinsize:cmdhist:expand_aliases:extquote:force_fignore:hi
stappend:hostcomplete:interactive_comments:login_shell:progcomp:prompt
vars:sourcepath
```

```
BASH_ALIASES=()
BASH_ARGC=()
BASH_ARGV=()
BASH_CMDS=()
BASH_LINENO=()
BASH_SOURCE=()
BASH_VERSINFO=([0]="4" [1]="2" [2]="46" [3]="2" [4]="release" [5]="x86
_64-redhat
-linux-gnu")
BASH_VERSION='4.2.46(2)-release'
COLUMNS=80
DIRSTACK=()
EUID=1000
GROUPS=()
HISTCONTROL=ignoredups
HISTFILE=/home/sysadmin/.bash_history
HISTFILESIZE=1000
HISTSIZE=1000
HOME=/home/sysadmin
HOSTNAME=localhost
HOSTTYPE=x86_64
```

Some output has been omitted from the terminal above for brevity.

1.2 Step 2

In some cases, you will want to distinguish between environment and local variables. To display only the environment variables, execute the `env` command:

```
env
```

```
[sysadmin@localhost ~]$ env
HOSTNAME=localhost
SHELL=/bin/bash
TERM=xterm
HISTSIZE=1000
USER=sysadmin
LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:bd=4
0;33;01:cd=40;33;01:or=40;31;01:mi=01;05;37;41:su=37;41:sg=30;43:ca=30
;41:tw=30;42:ow=34;42:st=37;44:ex=01;32:*.tar=01;31:*.tgz=01;31:*.arc=
01;31:*.arj=01;31:*.taz=01;31:*.lha=01;31:*.lz4=01;31:*.lzh=01;31:*.lz
ma=01;31:*.tlz=01;31:*.txz=01;31:*.tzo=01;31:*.t7z=01;31:*.zip=01;31:*
```

```
.z=01;31:*.Z=01;31:*.dz=01;31:*.gz=01;31:*.lrz=01;31:*.lz=01;31:*.lzo=
01;31:*.xz=01;31:*.bz2=01;31:*.bz=01;31:*.tbz=01;31:*.tbz2=01;31:*.tz=
01;31:*.deb=01;31:*.rpm=01;31:*.jar=01;31:*.war=01;31:*.ear=01;31:*.sa
r=01;31:*.rar=01;31:*.alz=01;31:*.ace=01;31:*.zoo=01;31:*.cpio=01;31:*
.7z=01;31:*.rz=01;31:*.cab=01;31:*.jpg=01;35:*.jpeg=01;35:*.gif=01;35:
*.bmp=01;35:*.pbm=01;35:*.pgm=01;35:*.ppm=01;35:*.tga=01;35:*.xbm=01;3
5:*.xpm=01;35:*.tif=01;35:*.tiff=01;35:*.png=01;35:*.svg=01;35:*.svgz=
01;35:*.mng=01;35:*.pcx=01;35:*.mov=01;35:*.mpg=01;35:*.mpeg=01;35:*.m
2v=01;35:*.mkv=01;35:*.webm=01;35:*.ogm=01;35:*.mp4=01;35:*.m4v=01;35:
*.mp4v=01;35:*.vob=01;35:*.qt=01;35:*.nuv=01;35:*.wmv=01;35:*.asf=01;3
5:*.rm=01;35:*.rmvb=01;35:*.flc=01;35:*.avi=01;35:*.fli=01;35:*.flv=01
;35:*.gl=01;35:*.dl=01;35:*.xcf=01;35:*.xwd=01;35:*.yuv=01;35:*.cgm=01
;35:*.emf=01;35:*.axv=01;35:*.anx=01;35:*.ogv=01;35:*.ogx=01;35:*.aac=
01;36:*.au=01;36:*.flac=01;36:*.mid=01;36:*.midi=01;36:*.mka=01;36:*.m
p3=01;36:*.mpc=01;36:*.ogg=01;36:*.ra=01;36:*.wav=01;36:*.axa=01;36:*.
oga=01;36:*.spx=01;36:*.xspf=01;36:

MAIL=/var/spool/mail/sysadmin

PATH=/usr/local/bin:/bin:/usr/bin:/usr/local/sbin:/usr/sbin:/home/sysa
dmin/.loca

l/bin:/home/sysadmin/bin

PWD=/home/sysadmin

HISTCONTROL=ignoredups

SHLVL=1

HOME=/home/sysadmin

LOGNAME=sysadmin

LESSOPEN=||/usr/bin/lesspipe.sh %s

_=/bin/env
```

1.3 Step 3

Execute the following command to see an alternative method to display only the environment variables:

```
export -p
```

```
[sysadmin@localhost ~]$ export -p

declare -x HISTCONTROL="ignoredups"

declare -x HISTSIZE="1000"

declare -x HOME="/home/sysadmin"

declare -x HOSTNAME="localhost"

declare -x LESSOPEN="||/usr/bin/lesspipe.sh %s"

declare -x LOGNAME="sysadmin"

declare -x LS_COLORS="rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:d
o=01;35:bd

=40;33;01:cd=40;33;01:or=40;31;01:mi=01;05;37;41:su=37;41:sg=30;43:ca=
30;41:tw=30;42:ow=34;42:st=37;44:ex=01;32:*.tar=01;31:*.tgz=01;31:*.ar
```

```
c=01;31:*.arj=01;31:*.taz=01;31:*.lha=01;31:*.lz4=01;31:*.lzh=01;31:*.
lzma=01;31:*.tlz=01;31:*.txz=01;31:*.tzo=01;31:*.t7z=01;31:*.zip=01;31
:*.z=01;31:*.Z=01;31:*.dz=01;31:*.gz=01;31:*.lrz=01;31:*.lz=01;31:*.lz
o=01;31:*.xz=01;31:*.bz2=01;31:*.bz=01;31:*.tbz=01;31:*.tbz2=01;31:*.t
z=01;31:*.deb=01;31:*.rpm=01;31:*.jar=01;31:*.war=01;31:*.ear=01;31:*.
sar=01;31:*.rar=01;31:*.alz=01;31:*.ace=01;31:*.zoo=01;31:*.cpio=01;31
:*.7z=01;31:*.rz=01;31:*.cab=01;31:*.jpg=01;35:*.jpeg=01;35:*.gif=01;3
5:*.bmp=01;35:*.pbm=01;35:*.pgm=01;35:*.ppm=01;35:*.tga=01;35:*.xbm=01
;35:*.xpm=01;35:*.tif=01;35:*.tiff=01;35:*.png=01;35:*.svg=01;35:*.svg
z=01;35:*.mng=01;35:*.pcx=01;35:*.mov=01;35:*.mpg=01;35:*.mpeg=01;35:*
.m2v=01;35:*.mkv=01;35:*.webm=01;35:*.ogm=01;35:*.mp4=01;35:*.m4v=01;3
5:*.mp4v=01;35:*.vob=01;35:*.qt=01;35:*.nuv=01;35:*.wmv=01;35:*.asf=01
;35:*.rm=01;35:*.rmvb=01;35:*.flc=01;35:*.avi=01;35:*.fli=01;35:*.flv=
01;35:*.gl=01;35:*.dl=01;35:*.xcf=01;35:*.xwd=01;35:*.yuv=01;35:*.cgm=
01;35:*.emf=01;35:*.axv=01;35:*.anx=01;35:*.ogv=01;35:*.ogx=01;35:*.aa
c=01;36:*.au=01;36:*.flac=01;36:*.mid=01;36:*.midi=01;36:*.mka=01;36:*
.mp3=01;36:*.mpc=01;36:*.ogg=01;36:*.ra=01;36:*.wav=01;36:*.axa=01;36:
*.oga=01;36:*.spx=01;36:*.xspf=01;36:"

declare -x MAIL="/var/spool/mail/sysadmin"

declare -x OLDPWD

declare -x PATH="/usr/local/bin:/bin:/usr/bin:/usr/local/sbin:/usr/sbi
n:/home/sy

sadmin/.local/bin:/home/sysadmin/bin"

declare -x PWD="/home/sysadmin"

declare -x SHELL="/bin/bash"

declare -x SHLVL="1"

declare -x TERM="xterm"

declare -x USER="sysadmin"
```

1.4 Step 4

To set the `PS1` local variable, execute the following command. This local variable will change the prompt to display the history number of the current command and will display the prompt character as `$`:

```
PS1='\# \$ '
```

```
[sysadmin@localhost ~]$ PS1='\# \$ '

5 $
```

The number `5` shown reflects a running count of the commands in the command history. This number will increase with each command that is executed. The number shown in your own window may vary, depending on how many commands you have executed since starting the shell.

```
ls /
```

```
5 $ ls /


 bin  etc   lib    media  opt   root  sbin  sys  usr
```

```
dev  home  lib64  mnt   proc  run   srv   tmp  var
6 $ ls

Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Vid
eos
7 $
```

1.5 Step 5

Set the `PS1` local variable so that it will show the current user name, hostname, the working directory, and the `$` prompt character by executing the following command:

```
PS1='\u@\h \w \$ '
```

```
7 $ PS1='\u@\h \w \$ '
sysadmin@localhost ~ $
```

1.6 Step 6

Before you change the `COLUMNS` environment variable, notice how this variable affects the number of columns output by commands like `ls /bin`:

```
ls /bin
```

```
sysadmin@localhost ~ $ ls /bin
[                                   objcopy
a2p                                 objdump
addr2line                           od
alias                               oldfind
applydeltarpm                       p11-kit
apropos                             package-cleanup
ar                                  passwd
arch                                paste
as                                  pathchk
awk                                 pcre-config
base64                              perl
basename                            perl5.16.3
bash                                perlbug
bashbug                             perldoc
bashbug-64                          perlthanks
bg                                  pgawk
```

```
bootctl                          pgrep
busctl                           pic
c++filt                          piconv
c2ph                             pinentry
c89                              pinentry-curses
c99                              ping
ca-legacy                        ping6
cal                              pinky
captoinfo                        pk12util
cat                              pkg-config
catchsegv                        pkill
catman                           pl2pm
cc                               pldd
cd                               pmap
certutil                         pod2html
chacl                            pod2man
chage                            pod2text
chardetect                       pod2usage
chcon                            post-grohtml
chfn                             pr
chgrp                            pre-grohtml
chmem                            preconv
chmod                            printenv
chown                            printf
chrt                             prlimit
chsh                             ps
cksum                            psed
clear                            pstruct
...
```

Some output has been omitted from the terminal above for brevity.
The partial output shown nearly fills the terminal width because the default value of the `COLUMNS` variable is `80`.

1.7 Step 7

Change the `COLUMNS` variable to a value of `60`, and use the `export` command to make the variable an environment variable:

```
export COLUMNS=60
```

```
sysadmin@localhost ~ $ export COLUMNS=60
```

1.8 Step 8

Now that you have changed the `COLUMNS` environment variable, you can see a difference in the number of columns output by executing the following command:

```
ls /bin
```

```
sysadmin@localhost ~ $ ls /bin


[
a2p
addr2line
alias
applydeltarpm
apropos
ar
arch
as
awk
base64
basename
bash
bashbug
bashbug-64
bg
bootctl
busctl
c++filt
c2ph
c89
c99
ca-legacy
```

```
cal
captoinfo
cat
...
```

Some output has been omitted from the terminal above for brevity.
The output displayed above shows that fewer columns are output because the `COLUMNS` environment variable limits the number of characters that can be displayed on a single line.

1.9 Step 9

You can also display the value of an individual variable by using the `echo` command along with the name of the variable prefixed with a `$` (dollar sign). This works for both local and environment variables. Execute the following command to display the value of the `COLUMNS` variable:

```
echo $COLUMNS
```

```
sysadmin@localhost ~ $ echo $COLUMNS
60
```

When displaying the value of a variable, preface the variable with the `$` character. When setting a variable, do not use the `$` character.

1.10 Step 10

Display the `PATH` environment variable, which determines which directories will be searched for commands executed from the command line, by executing the following command:

```
echo $PATH
```

```
sysadmin@localhost ~ $ echo $PATH
/usr/local/bin:/bin:/usr/bin:/usr/local/sbin:/usr/sbin:/home/sysadmin/
.local/b
```

1.11 Step 11

To test the `PATH` variable, first create a new directory in your home directory named `scripts` by executing the following command:

```
mkdir scripts
```

```
sysadmin@localhost ~ $ mkdir scripts
sysadmin@localhost ~ $ ls
Desktop    Downloads  Pictures  Templates  scripts
Documents  Music      Public    Videos
```

1.12 Step 12

Create a script file named `today` by executing the following commands:

```
echo '#!/bin/bash' > scripts/today
echo 'echo Today is $(date +%D)' >> scripts/today
cat scripts/today
```

```
sysadmin@localhost ~ $ echo '#!/bin/bash' > scripts/today
sysadmin@localhost ~ $ echo 'echo Today is $(date +%D)' >> scripts/tod
ay
sysadmin@localhost ~ $ cat scripts/today
#!/bin/bash
echo Today is $(date +%D)
```

Each `echo` command is taking the single-quoted text that immediately follows it and redirects that output to the `scripts/today` file. When a single `>` (greater than sign) follows the text, then that text overwrites any existing text in the file. When a double `>` (greater than sign) follows the text, then that text is added to the end of the existing text in the file.

1.13 Step 13

The `bash` command can be used to execute the `today` script by providing it with the path to the script file. Execute the following command to execute this script:

```
bash scripts/today
```

```
sysadmin@localhost ~ $ bash scripts/today
Today is 10/08/25
```

1.14 Step 14

If you want to be able to execute the script directly (without the `bash` command), the permissions for the user must include the read and execute permission. Otherwise, a message about denied permission will be shown. By default, files that are created are never automatically given execute permission. Attempt to execute the script, realizing that it lacks execute permission, by typing the path to the file, `scripts/today`:

```
scripts/today
```

```
sysadmin@localhost ~ $ scripts/today
-bash: scripts/today: Permission denied
```

1.15 Step 15

Viewing the detailed listing of the file with the `ls` command will allow you to see the permissions of the file. Use `ls -l` to view the details of the script file:

```
ls -l scripts/today
```

```
sysadmin@localhost ~ $ ls -l scripts/today
-rw-rw-r-- 1 sysadmin sysadmin 38 Oct  8 02:20 scripts/today
```

In the example output above, the user owner has *rw (read and write)* permissions, the group owner has the same, and the others have only *read* permission. Everyone lacks the necessary *execute* permission to run the file directly.

Note

Note the second through tenth characters of the output. The second, third, and fourth characters represent the permissions belonging to the user owner of the file (you, in this case).

```
-rw-rw-r-- 1 sysadmin sysadmin 39 Oct 13 18:43 scripts/today
```

The fifth, sixth, and seventh characters represent the permissions belonging to the members of the group that owns the file (if they are not the user owner of it).

```
-rw-rw-r-- 1 sysadmin sysadmin 39 Oct 13 18:43 scripts/today
```

The eighth, ninth, and tenth characters show the permissions for any user who is not the owner of the file, nor a member of the group that owns the file.

```
-rw-rw-r-- 1 sysadmin sysadmin 39 Oct 13 18:43 scripts/today
```

For the three characters that represent the permissions, if they show `rwx`, then that file has *read, write,* and *execute* permission. If a dash – character is shown instead of the `r`, then the entity lacks the *read* permission. Likewise, if a – is shown instead of the `w`, then the entity lacks the *write* permission. Of special interest in this case, if a – is shown instead of the `x`, then the entity has no *execute* permission.

1.16 Step 16

Use the `chmod` command to change the permissions on the script so anyone with access to the scripts directory would be able to execute the script directly. Execute the following commands:

```
chmod a+x scripts/today
ls -l scripts/today
```

```
sysadmin@localhost ~ $ chmod a+x scripts/today
sysadmin@localhost ~ $ ls -l scripts/today
-rwxrwxr-x 1 sysadmin sysadmin 38 Oct  8 02:20 scripts/today
```

Now that the fourth, seventh, and tenth characters of the output are an `x`, this indicates that the file is executable for the user, group, and others. The green color that is shown for the path to the file is another clue that indicates the file is executable.

1.17 Step 17

With the script file executable, it can now be run with any valid relative path or absolute path. Execute the following commands to verify that this script is now executable:

```
scripts/today
cd scripts
./today
/home/sysadmin/scripts/today
~/scripts/today
```

```
sysadmin@localhost ~ $ scripts/today
Today is 10/08/25
sysadmin@localhost ~ $ cd scripts
sysadmin@localhost ~/scripts $ ./today
Today is 10/08/25
sysadmin@localhost ~/scripts $ /home/sysadmin/scripts/today
Today is 10/08/25
sysadmin@localhost ~/scripts $ ~/scripts/today
Today is 10/08/25
```

The path that starts with `./` represents a relative path to the file in the current directory. The path that starts with `~/` represents a path to the user's home directory.

1.18 Step 18

Although relative and absolute paths work to execute the script, using the script name only will fail because the `scripts` directory is not listed as a value in the `PATH` variable. Try to execute the script by only using the script name; it will fail because the current directory is not searched for commands:

```
today
```

```
sysadmin@localhost ~/scripts $ today
-bash: today: command not found
```

1.19 Step 19

Display the `PATH` variable again to confirm that the `scripts` directory is not contained within it. Modify the `PATH` variable so that it contains the `scripts` directory and display the `PATH` variable to verify the change:

```
echo $PATH
PATH=$PATH:$HOME/scripts
echo $PATH
```

```
sysadmin@localhost ~/scripts $ echo $PATH
/usr/local/bin:/bin:/usr/bin:/usr/local/sbin:/usr/sbin:/home/sysadmin/
.local/bin:/home/sysadmin/bin
sysadmin@localhost ~/scripts $ PATH=$PATH:$HOME/scripts
sysadmin@localhost ~/scripts $ echo $PATH
/usr/local/bin:/bin:/usr/bin:/usr/local/sbin:/usr/sbin:/home/sysadmin/
.local/
```

1.20 Step 20

Once the `PATH` variable contains the path to the directory where your script is located, you can have it execute by typing only the name of the script file, regardless of your location within the filesystem. Execute the script from your current directory. Then change to your home directory by executing the `cd` command and execute the today script again to see this work:

```
today
cd
today
```

```
sysadmin@localhost ~/scripts $ today
Today is 04/16/25
sysadmin@localhost ~/scripts $ cd
sysadmin@localhost ~ $ today
Today is 10/08/25
```

1.21 Step 21

By design, changes to variables made in the shell do not persist after the user logs out of the system. Type `exit` to log out of the system:

```
exit
```

```
sysadmin@localhost ~ $ exit
logout
This lab has two user accounts (username :: password )

   root     :: netlab123
   sysadmin :: netlab123

Press the [Enter] key to begin...

Last login: Wed Oct  9 16:50:10 UTC 2019 on console
[sysadmin@localhost ~]$
```

Important

The virtual machine will need to be reset to continue. It will automatically log the user back in as the `sysadmin` user after the reset is complete.

1.22 Step 22

Display the `PATH` variable to confirm that its value has reverted back to the original list of directories:

```
echo $PATH
```

```
[sysadmin@localhost ~]$ echo $PATH
/usr/local/bin:/bin:/usr/bin:/usr/local/sbin:/usr/sbin:/home/sysadmin/
.local/b
```

1.23 Step 23

For a non-privileged user, the `.bash_profile` file can be used to customize the variables such as `PATH`. Execute the following command to see how the `PATH` variable is customized on line 10:

```
cat -n .bash_profile
```

```
[sysadmin@localhost ~]$ cat -n .bash_profile
     1  # .bash_profile
     2
     3  # Get the aliases and functions
```

```
 4  if [ -f ~/.bashrc ]; then
 5          . ~/.bashrc
 6  fi
 7
 8  # User specific environment and startup programs
 9
10  PATH=$PATH:$HOME/.local/bin:$HOME/bin
11
12  export PATH
```

On line 10, the `PATH` variable is set to the value of the existing `PATH` variable and the `$HOME/bin` directory. Notice the colon `:` character that separates the paths.

1.24 Step 24

Use the `nano` editor to modify the `.bash_profile` file:

```
nano .bash_profile
```

```
[sysadmin@localhost ~]$ nano .bash_profile
  GNU nano 2.3.1                          File: .bash_profile


# .bash_profile


# Get the aliases and functions
if [ -f ~/.bashrc ]; then
        . ~/.bashrc
fi


# User specific environment and startup programs


PATH=$PATH:$HOME/.local/bin:$HOME/bin


export PATH
```

1.25 Step 25

Use the **Arrow Keys** to move to the 10th line and add `$HOME/scripts` to it:

```
Down Arrow (x9)
Right Arrow (or Ctrl+E)
:$HOME/scripts
```

```
  GNU nano 2.3.1                                File: .bash_profile


# .bash_profile


# Get the aliases and functions
if [ -f ~/.bashrc ]; then
        . ~/.bashrc
fi


# User specific environment and startup programs


PATH=$PATH:$HOME/.local/bin:$HOME/bin:$HOME/scripts


export PATH
```

1.26 Step 26

Exit the `nano` editor by pressing **Ctrl+X**. Type y to save the changes, and press the **Enter** key:

```
Ctrl+X
y
Enter
```

1.27 Step 27

Instead of logging out and logging back in to test the changes to the `.bash_profile`, which is processed when the Bash shell starts during the login process, the file can be processed by using the `source` command. In some cases, you might see the use of the `.` command. The `.` command is the same as the `source` command. Execute the following command:

```
. .bash_profile
```

```
[sysadmin@localhost ~]$ . .bash_profile
[sysadmin@localhost ~]$
```

Important

It is much safer to test changes to Bash initialization files by sourcing those files instead of logging out and back in again. Potentially, an error in changing one of the initialization files might prevent a user from being able to log in again. If there is an error in sourcing the file while you are logged in, then the problem can be easily corrected.

1.28 Step 28

Display the `PATH` to verify that it includes the `scripts` directory now that you have sourced the `~/.bash_profile` file:

```
echo $PATH
```

```
[sysadmin@localhost ~]$ echo $PATH
/usr/local/bin:/bin:/usr/bin:/usr/local/sbin:/usr/sbin:/home/sysadmin/
.local/bin:/home/sysadmin/bin:/home/sysadmin/.local/bin:/home/sysadmin
/bin:/home/sysadmin/scripts
```

The user's `$HOME/bin` (`/home/sysadmin/bin`) is included twice because it was added when the user first logged in and the `~/.bash_profile` file was processed, and again after sourcing the `~/.bash_profile` file. Having this path twice will not cause any problems.

1.29 Step 29

The files that are in the `/etc/skel` directory are automatically copied to a new user's home directory when their account is created. Execute the following command to view these files:

```
ls -a /etc/skel
```

```
[sysadmin@localhost ~]$ ls -a /etc/skel
.  ..  .bash_logout  .bash_profile  .bashrc
```

1.30 Step 30

To add user accounts or copy files into the `/etc/skel` directory requires the privileges of the superuser. Execute the following command to switch to the root user account. When prompted for the password, enter `netlab123`:

```
su -
```

```
[sysadmin@localhost ~]$ su -
Password:
Last login: Wed Oct  9 17:12:36 UTC 2019 on console
[root@localhost ~]#
```

1.31 Step 31

Copy the customized `.bash_profile` file from the home directory of the `sysadmin` user by executing the following command:

```
cp ~sysadmin/.bash_profile  /etc/skel
```

When prompted to overwrite the existing file, answer `y`:

```
[root@localhost ~]# cp ~sysadmin/.bash_profile  /etc/skel
cp: overwrite `/etc/skel/.bash_profile'? y
```

By placing this new `.bash_profile` into the `/etc/skel` directory, any new `BASH` account will automatically have the contents of this file placed in the home directory of the new user account. This allows the administrator the ability to create customizations that will affect all new user accounts.

1.32 Step 32

The `useradd` command can be used to create a new user account. Execute the following command to create a new user named `testuser`:

```
useradd testuser
```

```
[root@localhost ~]# useradd testuser
```

The `useradd` command will be covered in greater detail in a later module

1.33 Step 33

View the `.bash_profile` file in the `testuser` home directory to verify that the new `.bash_profile` file has been copied to the user's home directory. Execute the following command:

```
cat ~testuser/.bash_profile
```

```
[root@localhost ~]# cat ~testuser/.bash_profile
# .bash_profile


# Get the aliases and functions
if [ -f ~/.bashrc ]; then
        . ~/.bashrc
fi


# User specific environment and startup programs


PATH=$PATH:$HOME/.local/bin:$HOME/bin:$HOME/scripts


export PATH
```

1.34 Step 34

Because the `~/.bash_profile` file is processed only once, typically when the user logs in, it is a good place to put custom environment variable assignments since these variables only need to be created once per account. The `~/ .bashrc` file is processed every time a user starts a new bash shell, such as when opening a new terminal window.
The `~/.bashrc` is a good place to put aliases and functions that must be executed in order to be defined every time the user starts a new shell. View the `.bashrc` file in the `/etc/skel` directory to see what it currently does by executing the following command:

```
cat /etc/skel/.bashrc
```

```
[root@localhost ~]# cat /etc/skel/.bashrc
# .bashrc


# Source global definitions
if [ -f /etc/bashrc ]; then
        . /etc/bashrc
fi


# Uncomment the following line if you don't like systemctl's auto-pagi
ng feature
```

```
:
# export SYSTEMD_PAGER=


# User specific aliases and functions
```

Recall that this file will be copied to the home directory when creating a new user account. It is executed every time a new shell is opened. The code under `# Source global definitions` checks to see if there is an `/etc/bashrc`. If that file exists, then it will be sourced. This feature allows the system administrator a place to create aliases and functions that will be applied to all user accounts.

At the bottom of the file is a comment suggesting where user-specific aliases and functions should be added.

1.35 Step 35

Before adding aliases and functions to the `/etc/skel/.bashrc` file, it is recommended that you test them in an interactive shell to make sure that they work as you would expect. First, view the aliases that are currently defined in the shell by executing the `alias` command.

```
alias
```

```
[root@localhost ~]# alias
alias cp='cp -i'
alias egrep='egrep --color=auto'
alias fgrep='fgrep --color=auto'
alias grep='grep --color=auto'
alias l.='ls -d .* --color=auto'
alias ll='ls -l --color=auto'
alias ls='ls --color=auto'
alias mv='mv -i'
alias rm='rm -i'
alias which='alias | /usr/bin/which --tty-only --read-alias --show-dot
--show-
```

1.36 Step 36

To add an alias to your shell, use the `alias` command. Execute the following commands to define a new alias called `lt`, which will list files in reverse time order, and then display the current shell's aliases:

```
alias lt='ls -lrt'
alias
```

```
[root@localhost ~]# alias lt='ls -lrt'
[root@localhost ~]# alias
alias cp='cp -i'
alias egrep='egrep --color=auto'
alias fgrep='fgrep --color=auto'
alias grep='grep --color=auto'
alias l.='ls -d .* --color=auto'
alias ll='ls -l --color=auto'
alias ls='ls --color=auto'
alias lt='ls -lrt'
alias mv='mv -i'
alias rm='rm -i'
alias which='alias | /usr/bin/which --tty-only --read-alias --show-dot
--show-ti
```

1.37 Step 37

Make sure that the `lt` alias you defined does in fact list files in reverse by time. Execute the following command to list the `/var` directory in reverse time order:

```
lt /var
```

```
[root@localhost ~]# lt /var
total 72
drwxr-xr-x 2 root root 4096 Apr 11  2018 yp
drwxr-xr-x 2 root root 4096 Apr 11  2018 preserve
drwxr-xr-x 2 root root 4096 Apr 11  2018 opt
drwxr-xr-x 2 root root 4096 Apr 11  2018 nis
drwxr-xr-x 2 root root 4096 Apr 11  2018 local
drwxr-xr-x 2 root root 4096 Apr 11  2018 gopher
drwxr-xr-x 2 root root 4096 Apr 11  2018 games
drwxr-xr-x 2 root root 4096 Apr 11  2018 adm
lrwxrwxrwx 1 root root    6 Dec  4  2018 run -> ../run
lrwxrwxrwx 1 root root   11 Dec  4  2018 lock -> ../run/lock
```

```
lrwxrwxrwx 1 root root   10 Dec  4  2018 mail -> spool/mail
drwxr-xr-x 1 root root 4096 Jan 29  2019 kerberos
drwxr-xr-x 1 root root 4096 Mar 29  2019 spool
drwxr-xr-x 1 root root 4096 Mar 29  2019 empty
drwxr-xr-x 1 root root 4096 Apr 24 19:20 db
drwxr-xr-x 1 root root 4096 Jul  1 14:57 cache
drwxrwxrwt 1 root root 4096 Jul  1 14:57 tmp
drwxr-xr-x 1 root root 4096 Jul  1 14:57 log
drwxr-xr-x 1 root root 4096 Jul  1 14:58 lib
```

The dates and times are displayed in descending order in the output above. Your output may vary, but should be shown in reverse time order.

1.38 Step 38

To remove the alias definition from your current shell session, use the `unalias` command with the name of the alias. Execute the following commands to remove the `lt` alias and verify that it is removed:

```
unalias lt
alias
```

```
[root@localhost ~]# unalias lt
[root@localhost ~]# alias
alias cp='cp -i'
alias egrep='egrep --color=auto'
alias fgrep='fgrep --color=auto'
alias grep='grep --color=auto'
alias l.='ls -d .* --color=auto'
alias ll='ls -l --color=auto'
alias ls='ls --color=auto'
alias mv='mv -i'
alias rm='rm -i'
alias which='alias | /usr/bin/which --tty-only --read-alias --show-dot
--show-ti
```

1.39 Step 39

While aliases are typically shorter names for longer commands, functions allow you to execute multiple commands and work with arguments that are passed to these commands. To view the functions that are defined within your current Bash shell, execute the following command:

```
declare -f
```

```
[root@localhost ~]# declare -f
```

The lack of any output indicates that no functions are defined.

1.40 Step 40

A function can be defined by first typing the function name to be assigned, followed by a pair of parentheses, and then the commands and options of the function contained between a pair of curly braces. For example, the following function will list files in reverse by size:

```
lS() { ls -lrS $@; }
```

The function uses the `ls` command with the options `-lrS` to list files in reverse by size and the variable `$@` to refer to any arguments that might have been passed to the function. Execute the following commands to create the function and verify it exists (note, the function name is `lS`, a lowercase `l` and a capital `S`):

```
lS() { ls -lrS $@; }
declare -f
```

```
[root@localhost ~]# lS() { ls -lrS $@; }
[root@localhost ~]# declare -f
lS ()
{
    ls --color=auto -lrS $@
}
```

Recall that Linux is case-sensitive. As a result, `ls` and `lS` are different commands/functions.

1.41 Step 41

Check to make sure this new function performs properly, listing the files in descending order by size, by executing the following command:

```
lS /var
```

```
[root@localhost ~]# lS /var
total 64
lrwxrwxrwx 1 root root    6 Dec  4  2018 run -> ../run
lrwxrwxrwx 1 root root   10 Dec  4  2018 mail -> spool/mail
```

```
lrwxrwxrwx 1 root root   11 Dec  4  2018 lock -> ../run/lock
drwxr-xr-x 2 root root 4096 Apr 11  2018 yp
drwxrwxrwt 1 root root 4096 Jul  1 14:57 tmp
drwxr-xr-x 1 root root 4096 Mar 29  2019 spool
drwxr-xr-x 2 root root 4096 Apr 11  2018 preserve
drwxr-xr-x 2 root root 4096 Apr 11  2018 opt
drwxr-xr-x 2 root root 4096 Apr 11  2018 nis
drwxr-xr-x 1 root root 4096 Jul  1 14:57 log
drwxr-xr-x 2 root root 4096 Apr 11  2018 local
drwxr-xr-x 1 root root 4096 Jul  1 14:58 lib
drwxr-xr-x 1 root root 4096 Jan 29  2019 kerberos
drwxr-xr-x 2 root root 4096 Apr 11  2018 gopher
drwxr-xr-x 2 root root 4096 Apr 11  2018 games
drwxr-xr-x 1 root root 4096 Mar 29  2019 empty
drwxr-xr-x 1 root root 4096 Apr 24 19:20 db
drwxr-xr-x 1 root root 4096 Jul  1 14:57 cache
drwxr-xr-x 2 root root 4096 Apr 11  2018 adm
```

1.42 Step 42

The `unset` command can be used to remove a function's definition from within a shell. Execute the following commands to unset the function and verify that it no longer exists:

```
unset lS
declare -f
```

```
[root@localhost ~]# unset lS
[root@localhost ~]# declare -f
```

1.43 Step 43

After having tested the `lt` alias and the `lS` function, you are ready to make those available to users by adding them to the `.bashrc` file in the `/etc/skel` directory. Start the `nano` editor to edit this file by executing the following command:

```
nano /etc/skel/.bashrc
```

```
  GNU nano 2.3.1                    File: /etc/skel/.bashrc


# .bashrc


# Source global definitions
if [ -f /etc/bashrc ]; then
```

```
        . /etc/bashrc
fi


# Uncomment the following line if you don't like systemctl's auto-pagi
ng feature$
# export SYSTEMD_PAGER=


# User specific aliases and functions
```

1.44 Step 44

Move the cursor to the end of the last line of the file by using the **Arrow keys** on your keyboard. Then, type the alias and function definitions as follows:

```
Down Arrow (x11)
alias lt='ls -lrt'
lS() { ls -lrS $@; }
```

```
  GNU nano 2.3.1            File: /etc/skel/.bashrc
Modified


# .bashrc


# Source global definitions
if [ -f /etc/bashrc ]; then
        . /etc/bashrc
fi


# Uncomment the following line if you don't like systemctl's auto-pagi
ng featur$
# export SYSTEMD_PAGER=


# User specific aliases and functions
alias lt='ls -lrt'
lS() { ls -lrS $@; }
```

1.45 Step 45

Exit the `nano` editor by pressing **Ctrl+X**. Type y to save the changes and then press the **Enter** key:

```
Ctrl+X
y
Enter
```

1.46 Step 46

Test the new `/etc/skel/.bashrc` file by sourcing it. Execute the following command:

```
source /etc/skel/.bashrc
```

```
[root@localhost ~]# source /etc/skel/.bashrc
```

If any errors are shown, go back and fix them in the `/etc/skel/.bashrc` file and repeat this step.

1.47 Step 47

Verify that both the new alias and the new function are now defined within your shell by executing the following commands:

```
alias
declare -f
```

```
[root@localhost ~]# alias
alias cp='cp -i'
alias egrep='egrep --color=auto'
alias fgrep='fgrep --color=auto'
alias grep='grep --color=auto'
alias l.='ls -d .* --color=auto'
alias ll='ls -l --color=auto'
alias ls='ls --color=auto'
alias lt='ls -lrt'
alias mv='mv -i'
alias rm='rm -i'
alias which='alias | /usr/bin/which --tty-only --read-alias --show-dot
--show-tilde'
[root@localhost ~]# declare -f
lS ()
```

```
{
    ls --color=auto -lrS $@
}
```

Recall that the files in the `/etc/skel` directory only affect new user accounts. To affect existing users, you would either have to copy the alias and function definition into each user's `~./bashrc` file or place these definitions in the `/etc/bashrc` file.

This lab is complete.